PIM : Mini-projet 1

Raffinages **1**
Évaluation des raffinages par l'étudiant **2**
Remarques diverses **2**
Évaluation du code **3**

Raffinages

R0 : Jouer au jeu des 13 allumettes

R1 : Comment "Jouer au jeu**" ?**

Demander le niveau du robot	Level : **out**
Demander qui commence	Starter : **out**
Jouer	Grid : **in out**, Starter : **in out**
Afficher gagnant	

R2 : Comment "Demander le niveau du robot**" ?**

Écrire ("Niveau de l'ordinateur (n)aïf, (d)istrait, (r)apide ou (e)xpert ? ")
Lire (Level)
Déterminer le niveau du robot
Écrire ("Mon niveau est ")
Écrire (Level)

R2 : Comment "Demander qui commence**" ?**

Écrire ("Est-ce que vous commencez (o/n) ? ")
Lire (Starter)

R2 : Comment "Jouer**" ?**

```
Grid <- 13
Tant que Grid>0 alors
    Afficher jeu
    Jouer en fonction de la personne
    Grid <- Grid-Number
FinTantque
```

R2 : Comment “Afficher gagnant**” ?**

Si Starter = “o”

Écrire (“J'ai gagné.”)

Sinon

Écrire (“Vous avez gagné”)

FinSi

R3 : Comment “Déterminer le niveau du robot**” ?**

Selon Level **dans**

“N” **ou** “n” => Level <- naif

“D” **ou** “d” => Level <- distrait

“R” **ou** “r” => Level <- rapide

Autres => Level <- expert

FinSelon

R3 : Comment “Afficher jeu**” ?**

Pour I **de** 1 à 3 **faire**

Afficher la première ligne de barre avec la séparation toutes les 5

FinBoucle

R3 : Comment “Jouer en fonction de la personne**” ?**

Si Starter = “o” **or** Starter = “O” **alors**

Choisir nombre par joueur

Sinon

Choisir nombre par robot

FinSi

Starter<-”o”

Écrire (“Je prends” Number “allumettes.”)

R4 : Comment “Afficher la première ligne de barre avec la séparation toutes les 5**” ?**

Pour J **de** 1 à Grid **faire**

Écrire (“| “)

Faire le séparation

FinPour

R4 : Comment “Choisir nombre par joueur**” ?**

Écrire (“Combien d'allumettes prenez-vous ? ”)

Lire (Number)

Tant que not(Number <= Grid **et** 1<=Number<=3**) alors**

Afficher les différents messages en fonction de l’erreur

FinTantQue

Starter <- “n”

R4 : Comment "Choisir nombre par robot**" ?**

Selon Level **dans**

"naif" => Jouer en mode jeu naif

"distrait" => Jouer en mode jeu distrait

"rapide" => Jouer en mode jeu rapide

"expert" => Jouer en mode jeu expert

FinSelon

Starter <- "o"

R5 : Comment "Faire séparation**" ?**

Si J mod(5) = 0 **alors**

Écrire (" ")

FinSi

R5 : Comment "Afficher les différents messages en fonction de l'erreur**" ?**

Si Number > Grid **alors**

Écrire ("Arbitre : Il reste seulement" Grid "allumettes")

Lire (Number)

Et si Number>3 **alors**

Écrire ("Arbitre : Il est interdit de prendre plus de 3 allumettes.")

Lire (Number)

Sinon

Écrire ("Arbitre : Il faut prendre au moins une allumette.")

Lire (Number)

FinSi

R5 : Comment "Jouer en mode jeu naif**" ?**

Si Grid > 2 **alors**

Number <- Aléa(1,3)

Sinon

Number <- 3 --Il est impossible de changer le random

Obtenir une valeur correct

FinSi

R5 : Comment "Jouer en mode jeu distrait**" ?**

Number <- Aléa(1,3)

Tant que Number>Grid **alors**

Écrire ("Je prends" Number "allumettes.")

Écrire ("Arbitre : Il reste seulement" Grid "allumettes")

Number <- Aléa(1,3)

FinTantQue

R5 : Comment “Jouer en mode jeu rapide**” ?**

Si Grid > 2 **alors**

Number <- 3

Sinon

Number <- Grid

FinSi

R5 : Comment “Jouer en mode jeu expert**” ?**

Si Grid /= 1 **alors**

Tentative victoire

Sinon

Number <- 1

FinSi

R6 : Comment “Obtenir une valeur correcte” **?**

Tant que Number>Grid **alors**

Number <- Aléa(1,3)

FinTantQue

R6 : Comment “Tentative victoire**” ?**

Si Grid mod(4) = 1 **alors**

Number <- Aléa(1,3)

Sinon

Number <- 1

Obtenir valeur gagnante

FinSi

R7 : Comment “Obtenir valeur gagnante**” ?**

Tant que (Grid - Number) mod(4) /= 1 **alors**

Number <- Number + 1

FinTantQue

Évaluation des raffinages par l’étudiant

		Evaluation Etudiant (I/P/A/+)	Justification / commentaire	Evaluation Enseignant (I/P/A)
Forme (D-21)	Respect de la syntaxe Ri : Comment "... une action complexe ..." ? des actions combinées avec des structures de controle Rj : ...			
	Verbe à l'infinitif pour les actions complexes			
	Nom ou équivalent pour expressions complexes			
	Tous les Ri sont écrits contre la marge et espacés			
	Les flots de données sont définis			
	Une seule structure de contrôle par raffinage			
	Pas trop d'actions dans un raffinage (moins de 6)			
	Bonne présentation des structures de contrôle			
Fond (D21-D22)	Le vocabulaire est précis			
	Le raffinage d'une action décrit complètement cette action			
	Le raffinage d'une action ne décrit que cette action			
	Les flots de données sont cohérents			
	Pas de structure de contrôle déguisée			
	Qualité des actions complexes			

Remarques diverses

TODO : Indiquer ici ce qui est utile à l’enseignant pour comprendre les raffinages. Cette partie peut être vide.

Évaluation du code

		Consigne : Mettre O (oui) ou N (non) dans la colonne Etudiant suivant que la règle a été respectée ou non. Une justification peut être ajoutée dans la colonne “commentaire”.	
Commentaire	**Etudiant (O/N)**	**Règle**	**Enseignant (O/N)**
		Le programme ne doit pas contenir d'erreurs de compilation.	
		Le programme doit compiler sans messages d’avertissement.	
		Le code doit être bien indenté.	
		Les règles de programmation du cours doivent être respectées : toujours un Sinon pour un Si, pas de sortie au milieu d'une répétition...	
		Pas de code redondant.	
		On doit utiliser les structures de contrôle adaptées (Si/Selon/TantQue/Répéter/Pour)	
		Utiliser des constantes nommées plutôt que des constantes littérales.	
		Les raffinages doivent être respectés dans le programme.	
		Les actions complexes doivent apparaître sous forme de commentaires placés AVANT les instructions correspondantes, avec la même indentation	
		Une ligne blanche doit séparer les principales actions complexes	
		Le rôle des variables doit être explicité à leur déclaration (commentaire).	